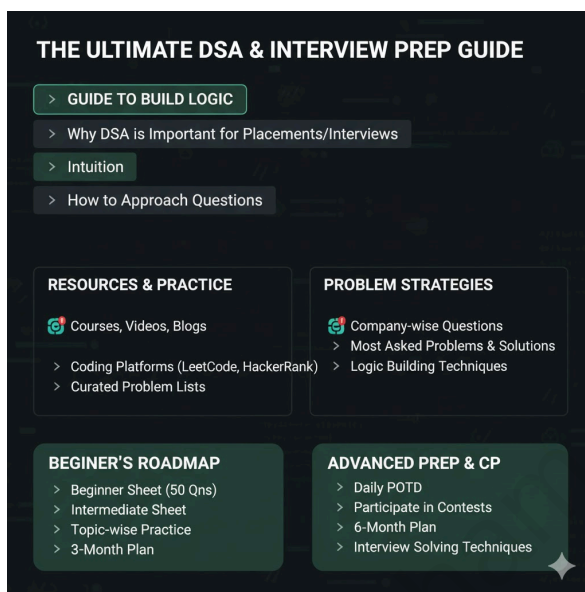
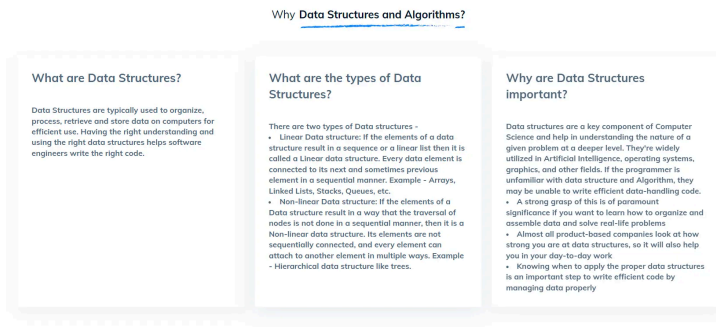


All DSA resources in one place



How to Start Learning DSA: A Simple Guide

The most important rule is to **just start**. Don't get confused by too many resources or think too much. Keep it simple.

Step 1: Choose a Language

First, pick one programming language you are comfortable with, like C++, Java, or Python. Master its basic syntax and concepts.

Step 2: Learn by Doing

The best way to learn DSA is by solving problems. **Code more to learn more**. The more you practice, the better you will get.

A Clear Roadmap

For Complete Beginners

- Start with a Beginner Sheet:** Find a curated list of about 50 essential DSA questions. This will build your foundation.—given below
- Move to an Intermediate Sheet:** Once you're comfortable, move on to a slightly more advanced sheet of problems.—given below
- Topic-wise Practice:** After completing the sheets, start solving problems topic-by-topic on platforms like LeetCode or GeeksforGeeks.

For Those with Some Experience

- Problem of the Day (POTD):** Make it a habit to solve the "Problem of the Day" on coding platforms. This keeps you consistent.
- Topic-wise Mastery:** Continue to solve problems organized by specific topics to strengthen your weak areas.
- Try Competitive Programming (CP):** Participate in coding contests. This will improve your problem-solving speed and ability to think under pressure.

👉 Most students think DSA is just for coding interviews.
But DSA is not just about getting the job,
It's about learning how to think.

Let me show you where it shows up in real life:

0. Stacks - Undo/Redo in editors, navigation history
1. Queues - Task scheduling, food delivery apps, customer service systems
2. Graphs - Google Maps, LinkedIn connections, web crawlers
3. Linked Lists - Media players, memory-efficient dynamic structures
4. Heaps / Priority Queues - Ride matching in Uber, CPU scheduling, real-time feeds
5. Trees - DOM structure in HTML, file systems, JSON parsing
6. Binary Search - Optimized searching, autocomplete, pagination logic
7. Recursion - File system traversal, game engines, tree operations
8. Sorting Algorithms - Search optimization, leaderboard ranking, ecommerce filters
9. Hash Tables - Fast lookups in databases, caching, language dictionaries
10. Tries - Auto-complete in search bars, contact list filtering, word games
11. Backtracking - Puzzle solving, UI pathfinding, constraint satisfaction problems
12. Sliding Window & Two Pointers - Real-time analytics, video buffering
13. Greedy Algorithms - Scheduling, resource allocation, Huffman encoding
14. Dynamic Programming - Finance apps, predictive modeling, optimization problems

And beyond code, DSA trains your brain to:

- Break big problems into manageable parts
- Handle complexity with structured thinking
- Develop patience, logic, and mental clarity
- Stay calm when brute force doesn't work

Learning DSA is like mental weightlifting.
It builds your problem-solving muscle.
which helps in interviews, tech jobs, and life decisions too.

So next time you're stuck on a problem, remind yourself:

You're not just learning DSA.
You're learning how to think like an engineer.

RESOURCES:

1.Books:

- Cracking the coding interview by Gayle Laakmann McDowell
- Data Structures and Algorithms Made Easy by Narshimha Karumanchi

2.COURSES AND YOUTUBE VIDEOS

- MIT OpenCourseWare: Introduction to Algorithms: <https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-006-introduction-to-algorithms-spring-2008/>
- Google DSA Course: Data Structures & Algorithms: https://techdevguide.withgoogle.com/paths/data-structures-and-algorithms/?programming_languages=c-plus-plus

Data structures playlist

Playlist of videos on data structures.

🔴 <https://youtube.com/playlist?list=PLDV1Zeh2NRsB6SWUrDFW2RmDotAfPbeHu&si=OTyKMDgtKHGz1WOQ>



Strivers A2Z-DSA Course | DSA Playlist | Placements

This playlist is not in C++ or Java or Python, it covers DSA and we write pseudocode, only one video is in C++, for the basics, but apart from that we cover ...

🔴 https://www.youtube.com/playlist?list=PLgUwDviBIf0oF6QL8m22w1hIDC1wJ_BHz



Complete C++ DSA Course | Data Structures & Algorithms Playlist

Let's start our DSA journey! Series will be consistent and will be completed ahead of schedule.

📺 https://www.youtube.com/playlist?list=PLfqMhTWNBT137L_EPQd34TsgV6IO55pt



Complete C++ Placement DSA Course

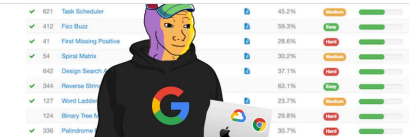
📺 <https://youtube.com/playlist?list=PLDzeHZWIZsTryvTXdMr6rPh4IDexB5NIA&feature=shared>



70 Leetcode problems in 5+ hours (every data structure) (full tutorial)

In this video we go through the solution and problem solving logic, walking through pretty much every leetcode question you need to know to pass a tech/programming interview, or to just become a better software engineer.

📺 <https://youtu.be/lvO88XxNAzs?si=ulF7DgJGFAS-d9Jc>



Competitive Programming/DSA Course | Hindi

Competitive Programming or DSA (Data Structure and Algorithms) Course For free. Today where a lot of students want to learn Competitive Programming and the v...

📺 <https://www.youtube.com/playlist?list=PLauivoElc3ggagradg8MfOZreCMmXMmJ->



Trees Playlist (Binary Tree | Binary Search Tree | AVL Tree)

All type of Trees

📺 https://www.youtube.com/playlist?list=PLQeARBv9gAFslul2ATmw7xP4eaQsJS_Lm



C++ STL | Competitive Programming

In this playlist I cover all the topic related to C++ STL like pairs, vectors, maps, sets and then I cover all the C++ STL algorithms which are very important...

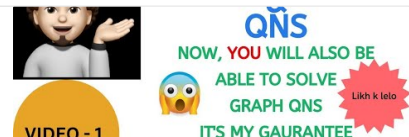
📺 <https://www.youtube.com/playlist?list=PLauivoElc3gh3RCiQA82MDI-gJfXQQVnn>



Graph Concepts & Qns : Convert Story To Code

The main motive of this playlist is to understand each Topic of Graph and solve qns on each of those topics to better understand and get a grip on those topl...

📺 <http://www.youtube.com/playlist?list=PLplkg80muX-LZB9jYzbbZchk277H5CbdY>



Dynamic Programming Playlist | Coding | Interview Questions | Tutorials | Algorithm

This playlist explains Dynamic Programming in a concise way. Explaining how to approach a Dynamic Programming problem and moreover how to identify it first.

📺 http://www.youtube.com/playlist?list=PL_z8CaSLPWekqhdCPmFohncHwz8TY2Go



3.BLOGS-GUIDES-GITHUBGS:

1. [20 Patterns to Crack MAANG](#)

2. [14 Tricks to Identify Patterns](#)

<https://drive.google.com/file/d/1eC55LL5fYSSn9Kd8llltevXTQWawOa10/view?usp=sharing>

[ALL ABOUT CODING ROUND SKILL TEST...pdf](#) - Google Drive

[How to prepare for tech interviews | Placement Series Day 4](#) - Google Drive

Links to Sites to help learn Data Structures and Algorithms Visually:

1. Data Structure Visualisations : <https://www.cs.usfca.edu/~galles/visualization/Algorithms.html>

2. Visualgo: <https://visualgo.net/en>

3. Visualizing Algorithms by Mike Bostock : <https://bost.ocks.org/mike/algorithms/>

GitHub - ashishps1/awesome-leetcode-resources: Awesome LeetCode resources to learn Data Structures and Algorithms and prepare for Coding Interviews.

Awesome LeetCode resources to learn Data Structures and Algorithms and prepare for Coding Interviews. - ashishps1/awesome-leetcode-resources

📺 https://github.com/ashishps1/awesome-leetcode-resources?mcp_token=eyJwaWQiOiE0MzIzNDY5InNpZCI6MTc3ODI4NTY1NywiYXgiOiIyNDI0YzAxZTJmMDMyNGY5ZjdmYWwRiNzk5OWY1MTM1NSIsInRlZjoxNzQyMDM4OTY4LjIleHAiOiE3NDQ0NTgxNjhh9.XaPjcl9tejPv8DpEtUQuvxtHLq4-pC1iwT_MBPQpoGHw&fbclid=PAY2xjawJCfW5leHRuA2FlbQixMAABphIFXODLndEI28LibS2t3qsxaEd2C_HUhgI3YGcNv9WzztnVPJEJrr5Cw_aem_FuOAQW05ogSiivLuTtxkJA

ashishp
leetcod

Awsome Leet
Structures and
Interviews.

9

visualising data structures and algorithms through animation - VisuAlgo

VisuAlgo was conceptualised in 2011 by Associate Professor Steven Halim (NUS School of Computing) as a tool to help his students better understand data structures and algorithms, by allowing them to learn the basics on their own and at their own pace. Together with his students from the National University of Singapore, a series of visualizations were developed and consolidated, from simple sorting algorithms to complex graph data structures. Though specifically designed for the use of NUS students taking various data structure and algorithm classes

 <https://visualgo.net/en>

LeetCode was HARD until I Learned these 15 Patterns

#21 - Patterns to master LeetCode

 <https://blog.algomaster.io/p/15-leetcode-patterns>



How I Spent 2 Hours Daily on DSA for 5 Years And I'm Still Going...

Best Way to Solve a DSA Problem When You're Stuck, (My DSA Revision Hack (That No One Talks About))

 <https://medium.com/@himanshushingour7/how-i-spent-2-hours-daily-on-dsa-for-5-years-and-im-still-going-46d4145303eb>

Coding Interview Patterns: The Smart, Data-Driven Way to Prepare for Tech Interviews

Coding interviews stressing you out? Get the structure you need to succeed. Get Interview Ready in 6 Weeks.

 <https://algo.monster/problems/stats>

takeuforward - Best Coding Tutorials for Free

takeuforward is the best place to learn data structures, algorithms, most asked coding interview questions, real interview experiences free of cost.

 <https://takeuforward.org/strivers-a2z-dsa-course/strivers-a2z-dsa-course-sheet-2>



Practice | GeeksforGeeks | A computer science portal for geeks

Platform to practice programming problems. Solve company interview questions and improve your coding intellect

 <https://www.geeksforgeeks.org/batch/gfg-160-problems/track/arrays-gfg-160/article/MTQ4NDk1>



Interview Preparation Kit

Level up your programming skills with our comprehensive guide containing interview questions, practice problems, coding challenges and alot more.

 <https://www.interviewbit.com/interview-preparation-kit/study-plan-3-months/>

Online Coding
Learn &
Practice



Top Interview 150 - Study Plan - LeetCode

Must-do List for Interview Prep

 <https://leetcode.com/studyplan/top-interview-150>



C++ Developer Roadmap

Master C++ for development with a focus on fundamentals and advanced C++ concepts. Build projects every step of the way to showcase your C++ developer skills.

 <https://www.codechef.com/roadmap/cpp-development>



Competitive Programming Roadmap - Become 5 Star on CodeChef

Elevate your CodeChef rating with our Competitive Programming Roadmap designed to guide you from basic logic skills to 5 star mastery. This roadmap includes curated problem sets, ranked contests, and technique drills across arrays, greedy, graphs, DP, and advanced topics. Whether you're just starting out or aiming for elite performance, our structured roadmap paves the

 <https://www.codechef.com/roadmap/become-5-star>



4.Challenges and all sheets

https://docs.google.com/spreadsheets/d/1mvlc8EYc3OVVU3X7NkoC0iZJr_45BL_pVxiJec0r94c/edit?gid=0#gid=0

CSES - CSES Problem Set - Tasks

<https://cses.fi/problemset/list>

Programming Skill Tests | Python, C, C++, Java, Data Structures & Algorithms

Test your knowledge in Python, C, C++, and Java. Get instant results and personalized recommendations to improve your coding skills. Start your tech career journey today!

 <https://www.codechef.com/skill-tests>



NeetCode

A better way to prepare for coding interviews.

 <https://neetcode.io/practice>

ProElevate

Your one-stop destination for DSA practice, interview experiences, and career opportunities.

 <https://www.proelevate.in/dsa-practice/arsh-dsa-sheet>

Programming - Interviewbit

Learn and Practice on almost all coding interview questions asked historically and get referred to the best tech companies

 <https://www.interviewbit.com/courses/programming/>



ProElevate

Your one-stop destination for DSA practice, interview experiences, and career opportunities.

 <https://www.proelevate.in/dsa-practice/6-companies-30-days>

AlgoMaster.io - Master LeetCode Patterns

Practice problems grouped by patterns and ace your technical interviews.

 <https://algomaster.io/practice/dsa-patterns>

[DSA practice roadmap1](#)

[DSA practice roadmap2](#)

[codestorywithMIK - YouTube Treasure.pdf](#)

[Leetcode 75 Questions \(NeetCode on yt\) - Sheet1.pdf](#)

☐ <https://github.com/chitrasaundane/DSA-14-days-Challenge>

Ashhad's DSA List

Ashhad Ahmad - Youtuber, Engineer.

 <https://ashhad.in/list/>

<https://prod-files-secure.s3.us-west-2.amazonaws.com/548d6b2f-5a61-415d-a7b4-aa8ee0048c3d/5288fca0-4cb2-4a51-b0d7-5324e5091bec/Untitled.pdf>

<https://prod-files-secure.s3.us-west-2.amazonaws.com/548d6b2f-5a61-415d-a7b4-aa8ee0048c3d/00fdb069-e831-4ad0-a5c0-ea25a3cc7bf1/Untitled.pdf>

<https://prod-files-secure.s3.us-west-2.amazonaws.com/548d6b2f-5a61-415d-a7b4-aa8ee0048c3d/c6dcfa3a-e68b-4494-87b4-a882a0901feb/Untitled.pdf>



Things to remember:

1. Follow a DSA sheet which has around 350-400 questions.
2. Don't simply do 'easy' difficulty problems.
3. Use ABOVE PAGES to solve DSA problems, it is user friendly-GFG >> LEETCODE (COZ IT WONT SHOW ANS EASILY).
4. If you're stuck on any topic, go to YouTube and clear your basics.
5. Don't worry if you're not able to build logic initially, it comes with practise.

Bonus : To build Confidence in DSA start giving Contest : on Leet-Code/ Hacker Rank Rank / Coding Ninjas / Geeks for Geeks and others



BEGINNER'S CODING SHEET—By Tharun
<https://www.linkedin.com/in/tharunbanda74/>

BEGINNER'S CODING SHEET

By Tharun

One-Stop Solution for All Newbies in Coding

Topics Covered:

1. Basic Input/Output
2. If-Else Statements
3. Loops (For, While, Do-While)
4. Switch Case & Patterns
5. Functions & Recursion
6. Arrays & Strings
7. Pointers & Memory Management
8. File Handling (Basics)
9. Debugging & Error Handling
10. Basic Object-Oriented Concepts

1. Basic Input/Output

1. Write a program to print "Hello, World!".
2. Write a program to take an **integer input** from the user and print it.
3. Write a program to add **two integers** entered by the user.
4. Write a program that takes **two numbers (dividend & divisor)** and prints the **quotient and remainder**.
5. Write a program to find the **size of int, float, double, and char** on your system.
6. Write a program to **swap two numbers** using a third variable.
7. Write a program to **swap two numbers without using a third variable**.
8. Write a program to find the **ASCII value** of a character entered by the user.

2. If-Else Statements

1. Write a program to check whether a **number is even or odd**.
2. Write a program to check whether a **character is a vowel or consonant**.
3. Write a program to find the **largest among three numbers** entered by the user.
4. Write a program that takes **coefficients of a quadratic equation** and finds its roots.
5. Write a program to check whether a **year is a leap year or not**.

3. Loops (For, While, Do-While)

1. Write a program to calculate the **sum of the first N natural numbers**.
2. Write a program to find the **factorial** of a given number.
3. Write a program to generate the **multiplication table** of a number.
4. Write a program to print the **Fibonacci series up to N terms**.
5. Write a program to **reverse a number**.
6. Write a program to find the **sum of digits** of a number.



Beginner to Intermediate Coding Sheet—By Tharun
<https://www.linkedin.com/in/tharunbanda74/>

Tharun's Beginner to Intermediate Coding Sheet

Introduction

This sheet is designed to help beginners build a solid foundation in coding, data structures, and algorithms. The goal is to take you from basic programming to understanding **recursion**, **building your own data structures**, and **solving real-world coding problems** to start **Competitive Programming (CP)**.

Basic Programming Questions

1. Basic Input/Output & Operations

1. Write a program to print "Hello, World!".
2. Write a program to take an integer input and print it.
3. Write a program to add two numbers entered by the user.
4. Write a program to find the quotient and remainder of division.
5. Write a program to find the size of `int`, `float`, `double`, and `char` in your system.
6. Write a program to swap two numbers using a temporary variable and without using a temporary variable.
7. Write a program to find the ASCII value of a character.
8. Write a program to multiply two decimal numbers.
9. Write a program to check whether a number is even or odd.
10. Write a program to check whether a character is a vowel or consonant.

2. If-Else & Loops

1. Find the largest number among three numbers entered by the user.
2. Check whether a given year is a leap year.
3. Find the sum of first N natural numbers.
4. Find the factorial of a number.
5. Generate the multiplication table of a number.
6. Display the Fibonacci series up to Nth term.
7. Find the GCD and LCM of two numbers.
8. Reverse a given number and find the sum of its digits.
9. Check whether a number is a palindrome.
10. Check whether a number is prime.

3. Pattern Problems

1. Print a solid rectangle pattern.
2. Print a hollow rectangle pattern.
3. Print a half-pyramid pattern.
4. Print an inverted half-pyramid pattern.
5. Print a full-pyramid pattern.
6. Print an inverted full-pyramid pattern.
7. Print Pascal's Triangle.
8. Print Floyd's Triangle.

4. Recursion (Basic to Medium)

1. Find the sum of N natural numbers using recursion.

7. Write a program to **check if a number is a palindrome**.
8. Write a program to check if a number is **prime**.
9. Write a program to print **prime numbers within a given range**.
10. Write a program to **find the LCM and GCD** of two numbers.

4. Switch Case & Patterns

1. Write a program to **implement a basic calculator** using the switch case.
2. Write a program to **display different star patterns**:
 - Solid Rectangle
 - Hollow Rectangle
 - Right-Angled Triangle
 - Inverted Pyramid
 - Pascal's Triangle

5. Functions & Recursion

1. Write a program to **find the sum of N natural numbers using recursion**.
2. Write a program to **calculate the factorial of a number using recursion**.
3. Write a program to **find GCD using recursion**.
4. Write a program to **calculate power of a number using recursion**.

6. Arrays & Strings

1. Write a program to take **N elements in an array** and find the average.
2. Write a program to find the **largest element in an array**.
3. Write a program to **reverse a string**.
4. Write a program to **check if a string is a palindrome**.
5. Write a program to count the **number of vowels and consonants in a string**.
6. Write a program to **sort an array in ascending order**.

7. Pointers & Memory Management

1. Write a program to access array elements using **pointers**.
2. Write a program to **swap two numbers using pointers**.
3. Write a program to **dynamically allocate memory** for an array and input values.
4. Write a program to **implement pointer arithmetic**.

8. File Handling (Basics)

1. Write a program to **create a file and write data into it**.
2. Write a program to **read data from a file** and print it.
3. Write a program to **append data to an existing file**.
4. Write a program to **count the number of lines in a file**.

9. Debugging & Error Handling

1. Write a program that **handles division by zero exception**.
2. Write a program that **handles an array out-of-bound error**.
3. Write a program to **debug a segmentation fault**.

10. Basic Object-Oriented Concepts (C++ & Java Users)

2. Find the factorial of a number using recursion.
3. Find GCD of two numbers using recursion.
4. Compute the power of a number using recursion.
5. Solve the Tower of Hanoi problem.
6. Generate all subsets of a set using recursion.
7. Solve the N-Queens problem using recursion.
8. Solve the Josephus problem using recursion.
9. Implement recursive DFS and BFS for a graph.
10. Find all permutations of a given string.

5. Building Data Structures from Scratch

Linked List

1. Implement a singly linked list with insertion, deletion, and traversal.
2. Implement a doubly linked list.
3. Reverse a linked list.
4. Detect and remove a cycle in a linked list.
5. Find the middle of a linked list.

Stacks and Queues

1. Implement a stack using arrays and linked list.
2. Implement a queue using arrays and linked list.
3. Implement a circular queue.
4. Implement a priority queue.
5. Implement a deque (double-ended queue).

Trees

1. Implement a binary tree with insert, delete, and traversal (Inorder, Preorder, Postorder).
2. Implement a binary search tree (BST).
3. Find the height of a binary tree.
4. Find the lowest common ancestor (LCA) of two nodes in a BST.
5. Convert a binary tree into a doubly linked list.

Graphs

1. Implement a graph using adjacency matrix and adjacency list.
2. Implement BFS and DFS traversal.
3. Detect a cycle in an undirected graph.
4. Find the shortest path in a graph using Dijkstra's algorithm.
5. Implement Floyd Warshall and Bellman-Ford algorithms.

6. Using Built-in Data Structures in Various Languages

Python

1. Use `list`, `set`, `dict`, `tuple` for different data operations.
2. Use `collections` module (`Counter`, `deque`, `defaultdict`).
3. Use `heapq` to implement a min-heap.
4. Use `itertools` for generating permutations and combinations.

C++ STL

1. Implement `vector`, `map`, `set`, `stack`, `queue`, and `priority_queue`.
2. Use `sort()` function with custom comparators.
3. Use `lower_bound()` and `upper_bound()`.

1. Write a program to define a **class and create an object**.
2. Write a program to **demonstrate encapsulation (private and public access specifiers)**.
3. Write a program to **implement inheritance**.
4. Write a program to **overload a function**.
5. Write a program to **overload an operator**.

Final Notes:

- This sheet is designed for **absolute beginners** and covers **all essential coding topics**.
- If you complete these problems, you will have a **strong foundation in programming**.
- Keep **practicing regularly** and explore **more advanced topics** as you progress.

 **Happy Coding!** 

👉 DSA → CP <https://www.tle-eliminators.com/cp-sheet>

you'll figure it out. But start with DSA, trust me on this It will save you. And here's how to do it:"

1 Just start.

Pick a programming language and begin solving problems. Don't waste time hunting for the perfect resource. Start anywhere.

2 Build confidence with easy problems.

Solve 30-40 simple questions first. Even if it's something basic, you'll feel like you belong.

3 Master the fundamentals.

Understand arrays, strings, recursion, and sorting thoroughly before moving on. Advanced topics like DP and Graphs will only make sense if your basics are solid.

4 Learn topic-wise first.

When learning something new, solve topic-specific problems. Once you're confident, mix in random problems to build intuition.

5 Struggle before seeking help.

When you get stuck, spend 20-30 minutes brainstorming before looking at solutions. That's where the real learning happens.

6 Understand why a solution works.

Knowing the proof behind a concept is more valuable than just memorizing the steps.

7 Push your limits.

Once you're comfortable, challenge yourself with harder problems. Progress happens when you step out of your comfort zone.

8 Track your time.

There's a big difference between solving a problem in 2 hours and 20 minutes. Work on speeding up as you improve.

9 Take part in contests.

Competitive programming is a great way to track your progress and build problem-solving under pressure.

10 It's about understanding, not numbers.

Don't chase problem counts. Focus on grasping the concepts behind each solution.

Java Collections

1. Use `ArrayList`, `LinkedList`, `HashMap`, `TreeMap`, `Stack`, and `Queue`.
2. Use `Collections.sort()` and custom comparators.

7. Real Coding Questions from Online Platforms

1. **Two Sum** - Find two numbers in an array that add up to a given target. (*LeetCode Easy*)
2. **Longest Substring Without Repeating Characters** - Find the length of the longest substring with unique characters. (*LeetCode Medium*)
3. **Valid Parentheses** - Check if a string of parentheses is valid. (*LeetCode Easy*)
4. **Merge Intervals** - Given a set of intervals, merge overlapping ones. (*LeetCode Medium*)
5. **Kth Largest Element in an Array** - Find the Kth largest element using heaps. (*LeetCode Medium*)
6. **Find the Duplicate Number** - Find the repeating number in an array. (*LeetCode Medium*)
7. **Binary Tree Level Order Traversal** - Print tree levels. (*LeetCode Medium*)
8. **Course Schedule** - Determine if all courses can be completed. (*LeetCode Medium, Graph*)
9. **LRU Cache Implementation** - Design an LRU cache. (*LeetCode Medium, Design*)
10. **Dijkstra's Algorithm** - Find shortest path in a graph. (*GeeksforGeeks, Graphs*)
11. **Implement Trie (Prefix Tree)** (*LeetCode Medium, Tries*)
12. **Word Ladder** - Find the shortest transformation sequence from one word to another. (*LeetCode Hard, Graphs*)
13. **Knapsack Problem** - Solve the 0/1 Knapsack problem. (*Dynamic Programming, GeeksforGeeks*)
14. **Minimum Spanning Tree** - Implement Kruskal's and Prim's algorithm. (*GeeksforGeeks, Graphs*)
15. **Subset Sum Problem** - Solve using recursion and DP. (*GeeksforGeeks, DP*)

8. Final Advice

- **Learn Python functions for all data structures.**
- **Master C++ STL and Java Collections for efficiency.**
- **Solve at least 300-400 problems before serious CP. but give contest in codechef and in codeforces as per your rating**
- **Practice coding on LeetCode, CodeForces, and AtCoder.**
- **Keep building real projects to reinforce learning!** 

5.PLATFORMS

Problems - LeetCode

Boost your coding interview skills and confidence by practicing real interview questions with LeetCode. Our platform offers a range of essential problems for practice, as well as the latest questions being asked by top-tier companies.

 <https://leetcode.com/problemset/>



NeetCode

A better way to prepare for coding interviews.

 <https://neetcode.io/practice>

Practice | GeeksforGeeks | A computer science portal for geeks

Platform to practice programming problems. Solve company interview questions and improve your coding intellect

 https://www.geeksforgeeks.org/explore?page=1&sortBy=submissions&utm_source=auth&utm_medium=profile&utm_campaign=empty-data



takeuforward - Best Coding Tutorials for Free

takeuforward is the best place to learn data structures, algorithms, most asked coding interview questions, real interview experiences free of cost.

 <https://takeuforward.org/strivers-a2z-dsa-course/strivers-a2z-dsa-course-sheet-2/>



CSES - CSES Problem Set - Tasks

<https://cses.fi/problemset/list>

Coding Ninjas Studio

 <https://www.codingninjas.com/studio/problems>

Code 360 by Coding Ninjas

 <https://www.naukri.com/code360/home>

Week 1 | Interview Preparation | HackerRank

Week 1 Prepare for you upcoming programming interview with HackerRank's Ultimate Interview Preparation Kit

 <https://www.hackerrank.com/interview/preparation-kits/three-month-preparation-kit/three-month-week-one/challenges>

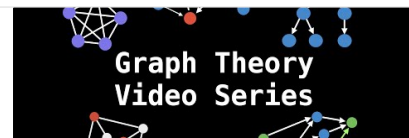


6. Some best of playlists topic wise

Graph Theory Playlist

A playlist of graph theory algorithm videos!

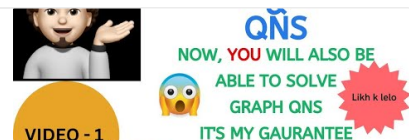
 <https://www.youtube.com/playlist?list=PLDV1Zeh2NRsDGO4--qE8yH72HFL1Km93P>



Graph Concepts & Qns : Convert Story To Code

The main motive of this playlist is to understand each Topic of Graph and solve qns on each of those topics to better understand and get a grip on those topic...

 <http://www.youtube.com/playlist?list=PLplkg8OmuX-LZB9jYzbbZchk277H5CbY>



Trees Playlist (Binary Tree | Binary Search Tree | AVL Tree)

All type of Trees

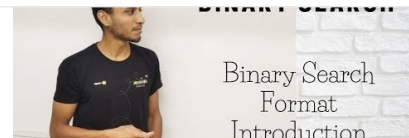
 https://www.youtube.com/playlist?list=PLQEA9BV9gAFslul2ATmw7xP4eaQsjS_Lm



Binary Search | Interview Questions | Coding | Tutorials | Algorithm

This playlist explains Binary Search in a concise way. Explaining how to approach a Binary Search problem and moreover how to identify it first.

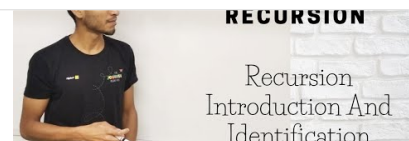
 http://www.youtube.com/playlist?list=PL_z_8CaSLPWeYfhtuKHj-9Mpyb6XQJ_f2



Recursion Playlist | Coding | Interview Questions | Algorithm | Tutorials

This playlist explains Recursion in a concise way. Explaining how to approach a Recursive problem and moreover how to identify it first.

 http://www.youtube.com/playlist?list=PL_z_8CaSLPWeT1ffjilmo0sYTcnLzo-wY



Backtracking

This playlist explains Backtracking in a concise way. Explaining how to approach a Backtracking problem and moreover how to identify it first.

http://www.youtube.com/playlist?list=PL_z_8CaSLPWdbOTog8Jxk9XOjzUs3egMP

BACKTRACKING Course Overview



DP Sheet By Shivam Bhadani

DP Sheet By Shivam Bhadani Conceptual Problems 0 - 1 Knapsack Problem Target Sum Coin Change Partition Equal Subset Sum Reducing Dishes Rod Cutting Unique Paths Unique Paths II Dungeon Game Longest Increasing Subsequence Box Stacking Russian Doll Envelopes Longest Common Subsequence Longest C...

<https://t.co/lxBKSw7C3K>



Dynamic Programming Playlist | Coding | Interview Questions | Tutorials | Algorithm

This playlist explains Dynamic Programming in a concise way. Explaining how to approach a Dynamic Programming problem and moreover how to identify it first.

http://www.youtube.com/playlist?list=PL_z_8CaSLPWekhdCPmFohncHwz8TY2Go

PROGRAMMING

DP Introduction



Tries Series - by Love Babbar

<https://www.youtube.com/playlist?list=PLDzeHZWIZsToGppbCLGKIYI-gTVASNEVb>



Maths Playlist | Language Independent Course

In this playlist, you will learn about Maths in depth. You will also find notes, and code for your specific language. The focus is to do this playlist without...

https://www.youtube.com/playlist?list=PLGwUwDvBif0oFON1SRGcMqMihIz4EXx_F



Two Pointer and Sliding Window Playlist | Language Independent Course

In this playlist, you will learn about 2 pointers and Sliding Window in depth. You will also find notes, and code for your specific language. The focus is to...

https://www.youtube.com/playlist?list=PLGwUwDvBif0q7vrFA_HEWcqRqMpCXzYAL



Bit Manipulation Playlist | Language Independent Course

In this playlist, you will learn Bit Manipulation in depth. You will also find notes, and code for your specific language. The focus is to do this playlist without...

<https://www.youtube.com/playlist?list=PLGwUwDvBif0rnqh8QsJaHyIX7KUiaPUv7>



Linked List | Beginner to Advanced for FAANG and PBC Interview Rounds

This series is language-independent, and we have used pseudocode in all the lectures. Please follow this sheet for a better learning experience: <https://take...>

<https://www.youtube.com/playlist?list=PLGwUwDvBif0rAuz8tVcM0AymmhTTrsfaLU>



Stack and Queue Playlist | Language Independent Series

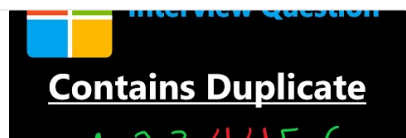
In this playlist, you will learn about Stack and Queue algorithms in depth. You will also find notes, and code for your specific language. The focus is to do...

<https://www.youtube.com/playlist?list=PLGwUwDvBif0pOd5zvVVSzgp06BaCpHT9c>



NeetCode 150

https://youtube.com/playlist?list=PLPe9lkX86X3y5m_MvtNu2ughxsvkqUNK&si=xGEqRIhpz7EsO3HR



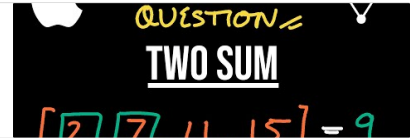
Leetcode BLIND-75 Solutions

https://youtube.com/playlist?list=PLot-Xpze53ldVwtstag2TL4HQhAnC8ATf&si=pk-m_RuS-4WIMdQh



Blind 75

 https://youtube.com/playlist?list=PL2HJG3tFqkg8jMqGYtkWDpF53rGvtPa6V&si=Wld9v1_3hGQYunYd



GitHub - ashishps1/awesome-leetcode-resources: Awesome LeetCode resources to learn Data Structures and Algorithms and prepare for Coding Interviews.

Awesome LeetCode resources to learn Data Structures and Algorithms and prepare for Coding Interviews. - ashishps1/awesome-leetcode-resources

 https://github.com/ashishps1/awesome-leetcode-resources?mcp_token=eyJwaWQlOjE0MzIzNDYsInNpZCI6MTc3ODI4NTY1NywiYXgiOiIyNDI0YzAxZTJmMDMyNGY5ZjdmYW RiNzk5OWY1MTM1NSIsInRlZjoxNzQyMDM4OTY4LCJleHAiOjE3NDQ0NTgxNjhh9.XaPjcl9tejPv8DpEtUQuvvhLq4-pCtiwT_MBpQpoGHw&fbclid=PAY2xjawJCRw5leHRuA2FibQlxMA ABphlFXODLndEI28LibS2t3qsxaEd2C_HUhgig3YGCnV9WzztnVPUEJrr5Cw_aem_FuOAQw05ogSiivLuTtxkJA

ashishp
leetcode

Awsome Leet
Structures and
Interviews.

Rs 9

goutham sankeerth

Patterns

- [15 Leetcode Patterns](#)
- [20 DP Patterns](#)
- [Prefix Sum Pattern](#)
- [Top 'K' Elements Pattern](#)
- [Fast and Slow Pointers Pattern](#)
- [Kadane's Algorithm](#)
- [Two Pointers Pattern](#)
- [Linked List In-place Reversal Pattern](#)
- [Sliding Window Pattern](#)
- [Monotonic Stack Pattern](#)
- [Overlapping Intervals Pattern](#)
- [Backtracking Pattern](#)
- [Modified Binary Search Pattern](#)
- [Tree Patterns](#)
 - [Tree Iterative Traversal](#)
 - [Tree Question Pattern](#)
- [Graph Patterns](#)
- [DFS + BFS Patterns \(1\)](#)
- [DFS + BFS Patterns \(2\)](#)

Must-Read Leetcode Articles

- [Sliding Window Template](#)
- [Two Pointers Patterns](#)
- [Collections of Important String Questions](#)
- [Substring Problem Template](#)
- [Binary Search Template](#)
- [A General Approach to Backtracking Questions](#)
- [Monotonic Stack Template](#)
- [Heap Patterns](#)
- [Bit Manipulation Patterns](#)
- [Dynamic Programming Patterns](#)
- [Stock Series Patterns](#)

Curated Problems

- [AlgoMaster 300](#)
- [Blind 75](#)
- [Leetcode Top 100 Liked](#)
- [Leetcode Top Interview 150](#)

YouTube Playlist

- [AlgoMaster LeetCode Pattern Playlist](#)
- [Abdul Bari's Algorithms Playlist](#)
- [William Fiset's Data Structure Playlist](#)
- [William Fiset's Graphs Playlist](#)
- [Tushar Roy's Dynamic Programming Playlist](#)

Courses

- [Coursera - Algorithms, Part 1](#)
- [Coursera - Algorithms, Part 2](#)

Books

- [Data Structures And Algorithms Made Easy](#)



- Its not about solving , its all about how you solve , thought process , diff approach's to a problem
- Focus on core topics like **basics**, arrays, **graphs**, **DP**

Focus on Improvement

- Always attempt problems yourself first.
- Learn and refine your approach by reviewing solutions.
- Solve problems while strengthening concepts side-by-side.
- Maintain consistency in coding and learning.
 - Daily GitHub streak.
 - Daily Leet Code streak. [Leeco: The Mentor that you needed for Leetcode, Youtube, etc](#)

Skill Testing

- Take a skill test, MCQ test, mock exam, or certificate test.
- Attempt quizzes and participate in coding rounds.
- check out this in edge bookmarks—**present challenges-mock tests**

Some tips to ace your DSA Interviews

1. Write variables and function names descriptive. Don't use i, j, x, y etc.
2. If some code is repeating, never copy paste in two places. Wrap it in a reusable function and call that function multiple times.
3. Don't make the length of the code of your main function too big. Try to make different functions for small tasks and call it the main function.
4. Don't make too much complicated data types like pair<int, pair<int, int>>, instead make "struct" for it with descriptive names.
5. If you are stuck and can't think of any approach then ask for hints. Most interviewers don't mind giving small hints.
6. While writing the code don't write silently. Instead keep explaining each and every line while writing.
7. Don't waste time in writing brute force then optimized. Brute force don't have any marks in interviews. You will only get selected on the basis of your optimized solution. Brute solution will just waste your time for later questions. As soon as you get the question, you can tell the brute force approach verbally under 1 minute but after that spend all your time in explaining and coding optimized solution

How I Cracked the Coding Interview (In Under 3 Months)

Cracking the technical coding interview is one of the most important steps to becoming a software engineer, and I did it in under 3 months as a freshman in college. In this video I lay out a

 <https://www.youtube.com/watch?v=OmFL0CYeZ4I&t=25s>



How to Approach Questions

- **Read the Question Carefully:** Understand the constraints and requirements.
- **Dry Run:** Perform a dry run on paper to validate your approach- Perform a dry run with sample inputs.
- **Breakdown the Problem:** Analyze the given inputs and outputs step by step.—Analyze constraints and expected outputs.
- **Attempt the Question:**

- [Cracking the Coding Interview](#)

Newsletter

- [AlgoMaster Newsletter](#)

Visualization

- [VisuAlgo](#)
- [Algo-lens: Visualize leetcode problems](#)

LeetCode Extensions

- [LeetCode Timer](#): Easily time your leetcode practise sessions with automatic time setting based on difficulty.
- [LeetCode Video Solutions](#): Watch free LeetCode video ► solutions on the problem page itself.
- [LeetCode Format](#): Adds Format code button on leetcode to format the code using Prettier code formatter.
- [LeetHub v2](#): Automatically integrate your Leetcode & GeeksforGeeks submissions to GitHub.
- [LeetCode VS Code Extension](#): Solve LeetCode problems in VS Code.



DSA: BEYOND THE INTERVIEW
NOT JUST ABOUT THE JOB. IT'S ABOUT LEARNING HOW TO THINK

REAL-LIFE APPLICATIONS:

0. ↔ Stacks Undo/Redo, History
1. ⌚ Task Scheduling, Food Delivery
2. 🗺️ Graphs Google Maps, LinkedIn, Memory
3. 🚗 Heaps / Priority Queues, Real-time Feeds
5. 📁 HTML DOM, File Systems
6. 🔍 Binary Search, Pagination
7. 🌐 File Traversal, E-commerce Filters
8. 🎮 Sorting Game Engines
10. 🛡️ Puzzle Solving, UI atbatsins, Contact Filtering
12. 📊 Tries Table Algorithms
13. 📈 Sliding Window Analytics
14. 🧠 Dynamic Algorithm, Scheduling Modelling

DSA TRAINS YOUR BRAIN TO:

- ⚙️ Break big problems into manageable parts
- 📦 Handle complexity with
- 📦 Handle complexity with structured thinking
- 💡 Develop patience, logic, and mental clarity
- 🧩 Dazzle Solving, UI Pathfinding
- ⚡ Stay calm when brute force doesn't work
- 😊 Finance Apps, Predictive Modeling

- Spend **30-45 minutes** trying to solve the problem independently.
- Write down your logic on paper before coding.
- Implement the code while keeping the constraints in mind.
- Optimize your solution after completing the initial implementation.

• Review Solutions:

- If stuck, check solutions from trusted sources.
- Compare your solution with the **top 3 optimized solutions** and note improvements.
- do code yourself until you feel confident that you can code really
- Avoid copying solutions directly; understand and implement them on your own.—Implement the solution yourself until you feel confident.
- Note down mistakes and learnings—Focus on optimizing the solution.
- Use AI (like ChatGPT) only for syntax issues or small hints but not for solving the entire problem.

• Mark Difficult Questions:

- Note down questions you struggled with and revisit them later.

Why DSA is Important for Placements/Interviews:

Importance of DSA (Data Structures and Algorithms)

1. Core of Problem Solving

• Efficiency in Solving Problems:

- DSA knowledge helps optimize problem-solving by choosing the right data structure (e.g., arrays, strings, trees, graphs) and algorithms for specific scenarios.
- It focuses on reducing time complexity, space usage, and overall system efficiency.

• Interview Perspective:

- Interviewers assess how candidates approach and solve real-world problems using logical thinking and structured methods, reflecting how they can handle applications or systems in the job role.

2. Logical Thinking and Approach

- DSA builds logical reasoning and analytical skills.
- Candidates with strong DSA knowledge can derive efficient solutions for complex problems, focusing on clear workflows and minimizing resource usage.
- Those without DSA knowledge often take more time, need more resources, and fail to optimize solutions effectively.

3. Optimization in Real-Life Scenarios

- DSA is crucial for managing, storing, and processing large datasets efficiently, such as in competitive programming and backend systems.
- It ensures that solutions use minimal memory and processing power while achieving the desired results.

4. Applications in Interviews and Competitive Programming

- DSA showcases a candidate's ability to structure solutions, select appropriate algorithms, and implement them effectively during coding challenges.
- Real-life applicability of DSA in solving software and system-level issues enhances employability.

Key Takeaways

- DSA is not just overhyped; it plays a significant role in building efficient systems and solving real-world problems.
- Its importance lies in preparing individuals to handle complexity, think logically, and demonstrate problem-solving expertise in technical interviews.

Would you like a more detailed breakdown of any particular point?

▼ intuition

The Right Way to Build Intuition for Problem Solving: A Comprehensive Guide

Understanding Intuition

Intuition in problem-solving is essentially a gut feeling - an immediate response from your unconscious brain based on past experiences and accumulated knowledge. Just like driving a vehicle where you don't calculate exact measurements but make decisions based on instinct, problem-solving intuition works similarly.

How Intuition Works in Problem Solving

When you have a strong knowledge base (for example, having solved 500 problems), approaching the 501st problem becomes different. Your brain automatically recognizes patterns and similarities with previously solved problems. This recognition helps you identify potential approaches quickly:

- You might notice similar data arrangements
- You recognize familiar constraint patterns
- You can identify when certain techniques (like binary search) might be applicable
- Your brain connects new problems to existing solutions in your knowledge base

The Development Process

Intuition isn't instantaneous - it develops over time through consistent practice and exposure to various problems. The process involves:

1. Building a strong knowledge base through solving numerous problems
2. Recognizing patterns across different problem types
3. Developing the ability to make quick connections between new problems and past experiences
4. Learning to trust your initial instincts while remaining open to alternative approaches

Advantages of Intuition-Based Problem Solving

In time-bound situations (like coding interviews or competitive programming), intuition becomes particularly valuable because:

- It reduces the need for extensive analysis of every possible test case
- Eliminates the requirement to draw out complete solutions (like recursive call stacks) for every problem
- Allows for faster initial approach selection
- Helps in quick pattern recognition and solution framework identification

The Balance Between Intuition and Analysis

While intuition is valuable, it's important to understand its limitations:

- Intuition can sometimes lead to incorrect solutions, especially with greedy approaches
- Initial intuitive approaches might miss certain edge cases
- Some problems require deeper analysis despite strong intuitive feelings

Building Better Intuition: Practical Approaches

1. Categorized Learning

Take inspiration from resources like CP3 (Competitive Programming book), which categorizes dynamic programming problems into distinct patterns:

- Knapsack-style problems
- Bitmask DP problems
- Traveling salesman-type problems
- Other specific categories

Understanding these categories helps in quickly identifying which approach might work for new problems.

2. Pattern Recognition Training

- Pay attention to problem constraints
- Notice when certain values are suspiciously small or large

- Identify common problem structures
- Learn to recognize when specific data structures or algorithms are likely to be useful

3. Historical Understanding

Try to understand how algorithms were originally developed:

- For example, Heap Sort can be better understood by seeing it as an optimization of Selection Sort
- Understanding the evolution of solutions helps in developing problem-solving intuition

Practical Application Tips

1. Start with a solid foundation of basic problems
2. Gradually increase problem complexity
3. After solving each problem, reflect on:
 - What patterns did you notice?
 - How did your intuition help or mislead you?
 - What new patterns can you add to your knowledge base?

Developing Your Own Intuition Journey

To effectively build your problem-solving intuition:

1. Solve a variety of problems consistently
2. Maintain a record of patterns you encounter
3. Practice linking new problems to previously solved ones
4. Don't be afraid to trust your initial instincts while remaining open to alternative approaches
5. Learn from cases where your intuition was incorrect

Remember, building strong problem-solving intuition is a gradual process that requires consistent practice and patience. The key is to keep solving problems while consciously analyzing patterns and building your knowledge base.

When to Trust Your Intuition

- In time-constrained situations where quick decisions are needed
- When you recognize clear patterns from your past experience
- When initial pattern recognition strongly suggests a particular approach

When to Be Cautious

- With new types of problems you haven't encountered before
- When dealing with edge cases
- When your intuitive solution seems too simple for the given problem complexity

The journey to building strong problem-solving intuition is continuous and requires dedication to practice and learning. Keep solving problems, analyzing patterns, and building your knowledge base, and your intuition will naturally strengthen over time.

▼ build logic

1. Give More Time to Actually Writing Code

- **Why it's important:** Watching tutorials and videos about solving DSA problems can be helpful, but they do not provide the real coding experience. You will never truly understand the challenges involved in coding unless you practice writing the code yourself.
- **Actionable Tip:** Spend more time coding, debugging, and solving problems on your own. As you code, you will encounter errors that help you improve your problem-solving skills. Consistent practice helps you recognize patterns and improves your logic-building ability.
- **How to Practice:** Solve at least 2-3 problems every day. Start with easier problems and gradually move to medium and hard ones. Don't fall into the trap of solving just a few problems per day and then taking long breaks. Consistency is key.

2. Master the Basics

- **Why it's important:** Many people jump into platforms like LeetCode or CodeChef to practice without having a solid grasp of the basics. This often leads to frustration and a drop in motivation.
- **Actionable Tip:** Begin by strengthening your understanding of the basic data structures and algorithms. Solve easy problems first to ensure your foundation is strong.
- **How to Practice:** Utilize platforms like InterviewBit to clear your basics. Once your foundation is solid, you can move to more advanced problems. For practice, use platforms like CodeForces, LeetCode, or HackerRank.
- **Further Learning:** If you're also interested in data science, the video recommends Odeon School for specialized courses in data science, including SQL, Python, Machine Learning, and AI, to enhance your skills.

3. Identify Where You Get Stuck

- **Why it's important:** Getting stuck at certain points is a normal part of learning. The key is to recognize where you're facing difficulty and treat it as a learning opportunity.
- **Actionable Tip:** When you get stuck, don't get discouraged. Write down the problem you are facing and note your thought process. This will help you avoid repeating the same mistakes in the future.
- **Personal Example:** The speaker shares an example of struggling with a problem related to parentheses validation. After trying different approaches, he found that using a stack helped solve the problem, and this became an important learning point.
- **How to Practice:** Every time you get stuck, try to break down the problem and look for the specific point where you are confused. This will help you to build better problem-solving strategies.

4. Learn Common Problem Patterns

- **Why it's important:** In DSA, many problems share common patterns. Recognizing these patterns will make it easier to solve new problems.
- **Actionable Tip:** Familiarize yourself with common DSA problem patterns such as:
 - **Sliding Window:** Often used for subarray-related problems.
 - **Kadane's Algorithm:** Used for finding the maximum sum subarray.
 - **BFS/DFS:** Common in graph-related problems.
 - **Knapsack Problem:** A dynamic programming problem, often used when choosing items under constraints.
 - **Longest Common Subsequence (LCS)** and **Longest Increasing Subsequence (LIS):** Common in string-related problems.
- **How to Practice:** Once you recognize a problem pattern, try solving it using the pattern. This makes it easier to build logic and solve the problem efficiently.

5. Start with a Brute Force Solution

- **Why it's important:** Many coders jump straight into trying to find the optimal solution, but writing a brute force solution first can give clarity on the problem.
- **Actionable Tip:** Never hesitate to start with the brute force approach. While it may not be the most efficient, it helps in understanding the problem and provides a foundation on which you can build an optimized solution.
- **How to Practice:** Solve problems using brute force first, even if it's not optimal. Once you've solved it, focus on improving the efficiency of your solution.

6. Dry Run and Visualize

- **Why it's important:** Sometimes, when you're stuck on a problem, you can't think of an approach. In such cases, dry-running the problem can help visualize the solution.
- **Actionable Tip:** Take a pen and paper and draw out the problem. Avoid thinking about coding for now; focus on understanding how the solution works through visualizing inputs, outputs, and steps involved.
- **How to Practice:** Work through the problem step-by-step using the dry run method. This will allow you to spot patterns or new approaches that might not be obvious at first glance.

7. Teach Someone Else

- **Why it's important:** Teaching is one of the best ways to reinforce your learning. When you explain a concept to others, it helps you identify gaps in your understanding.
- **Actionable Tip:** Once you've solved a problem or understood a concept, teach it to someone else. This could be a friend, a relative, or even writing a blog or tutorial about it.
- **How to Practice:** Find someone willing to listen or explain the concept online. By teaching them, you'll not only reinforce your knowledge but also identify areas where you need more clarity.

Conclusion:

By consistently following these seven strategies, you will see significant improvements in your problem-solving and logic-building skills in DSA. Whether you're just starting or facing challenges, these tips provide a structured approach to overcoming obstacles and enhancing your coding abilities.

▼ how to do dsa for clearing interviews

How to Prepare for Coding Interviews (The Smarter Way)

Preparing for coding interviews is not just about solving 1000 LeetCode problems. It's about learning smartly, practicing effectively, and remembering what you've learned. Here are some methods that worked for me (and many others).



1. Use AI Tools as Your Interviewer

Don't just solve problems, test yourself like you're in a real interview.

- Ask AI to rate your solution and suggest improvements.
- Example prompt:
"Act as an interviewer at Meta SWE Interview. I'll share my solution for [problem]. Please rate it and ask me 3 follow-up questions."

👉 This helps you think beyond just solving — you'll learn how to **explain your solution**, defend it, and answer tricky follow-ups.

2. Use AI as Your Coding Buddy

Sometimes our solution fails for a hidden test case, and we keep scratching our head. That's where AI can save time.

- Ask: *"Why is my current solution failing for this test case?"*
- Or: *"Debug my code for this problem."*

👉 Instead of wasting hours, you get quick hints and corrections. Think of it like having a coding friend sitting next to you.

3. Build a Spaced Repetition Log for Hard Problems

We forget things fast if we don't revise. So keep a personal log of the **problems you find difficult**.

- Example log rule:
"If I can't solve it on Day 1, I'll revisit on Day 3, then Day 7, then after 2 weeks."
- The key is to recognize **patterns** (sliding window, dynamic programming transitions, graph traversal tricks, etc.) and train your brain to spot them quickly.

👉 Just like remembering formulas in math, remembering patterns makes solving new problems much easier.



4. Record Yourself While Solving

This feels awkward at first but is a **game-changer**.

- Set a 30–45 min timer.
- Open your camera (or just voice record).
- Explain your solution as if you're talking to an interviewer.

👉 Example: "Okay, so the problem is about finding the longest substring without repeating characters. My idea is to use the sliding window technique..."

When you re-watch, you'll notice where you:

- Got stuck
- Explained badly
- Missed edge cases

This improves both **thinking speed** and **communication** (which is super important in real interviews).

5. Build Mini Side Projects Using DSA Concepts

Why just solve random problems when you can also build something cool with the same knowledge?

- Example project 1: Build a **Trie autocomplete tool** (like search bar suggestions).
- Example project 2: Make a **Heap-powered task scheduler** (like a to-do app that picks tasks based on priority).

👉 These projects make your DSA learnings stick and also look **amazing in your portfolio**.

6. Do a Lot of Mock Interviews

Practice under pressure matters.

- Use free platforms like Pramp, Interviewing.io, or even pair up with a friend.
- Target: at least **10 mocks** before your real interview.

👉 You'll get used to explaining, handling silence, and thinking out loud — all things interviewers love.

Final Thoughts

At the end of the day, cracking coding interviews is about **clarity + practice + consistency**.

- AI helps you speed up learning.
- Logs help you remember patterns.
- Recording helps you explain better.
- Side projects make your skills real.
- Mock interviews give you confidence.

If you combine all of these, you'll not only solve problems but also **look like someone ready for the job**.

▼ topics to cover CHECKLIST

- ☐ 1. Programming Fundamentals
 - ☐ Variables, data types, and operators
 - ☐ Control flow (loops, conditionals)
 - ☐ Functions and modular programming
 - ☐ Input/output and file handling
- ☐ 2. Basic Data Structures:
 - ☐ Arrays and dynamic arrays
 - ☐ Linked lists (singly linked, doubly linked)
 - ☐ Stacks and queues (array-based, linked list-based)

- ☐ Hash tables (hash functions, collision handling)
- ☐ 3. Basic Algorithms:
 - ☐ Linear search
 - ☐ Binary search
 - ☐ Bubble sort
 - ☐ Insertion sort
 - ☐ Selection sort
- ☐ 4. Advanced Data Structures:
 - ☐ Trees (binary trees, binary search trees)
 - ☐ Balanced trees (AVL trees, red-black trees)
 - ☐ Heaps (binary heap, priority queue)
 - ☐ Graphs (representation, traversal)
- ☐ 5. Advanced Algorithms:
 - ☐ Divide and conquer (merge sort, quicksort)
 - ☐ Dynamic programming (Fibonacci sequence, knapsack problem)
 - ☐ Graph algorithms (breadth-first search, depth-first search)
 - ☐ Shortest path algorithms (Dijkstra's algorithm, Bellman-Ford algorithm)
- ☐ 6. Problem Solving and Practice:
 - ☐ Solving coding problems (LeetCode, HackerRank, Codeforces)
 - ☐ Algorithmic complexity analysis (time complexity, space complexity)
 - ☐ Problem-solving techniques (logical reasoning, algorithm design)
- ☐ 7. Algorithmic Techniques:
 - ☐ Recursion and backtracking
 - ☐ Greedy algorithms
 - ☐ Sliding window technique
 - ☐ Two-pointer technique
 - ☐ Divide and conquer strategies
- ☐ 8. Optimization and Advanced Topics:
 - ☐ Algorithm analysis (asymptotic notation)
 - ☐ Memoization and dynamic programming optimization
 - ☐ Space-time tradeoffs (tradeoff between time and space complexity)
 - ☐ Additional data structures (heaps, hash tables, advanced graph structures)
- ☐ 9. Practice Projects:
 - ☐ Implementing data structures and algorithms in practical projects
 - ☐ Solving real-world problems using algorithms
 - ☐ Optimizing algorithms for efficiency and performance
- ☐ 10. Continued Learning:
 - ☐ Staying updated on new algorithms and data structures
 - ☐ Reading books on algorithms and advanced topics
 - ☐ Exploring research papers and online resources for further learning

Pure DSA Algorithms

Binary Search Fast search in sorted array
 — Sliding Window — Check subarrays smartly
 Kadane's Algo — Max subarray sum
 — Merge Sort — Divide and sort
 — DFS/BFS Graph or grid traversal
 Prefix Sum — Fast range sum
 — Heap (Priority Queue) Get largest/smallest fast
 — Two Pointers — Solve with left-right movement
 — Union-Find (DSU) — Group tracking, cycle detection
 0/1 Knapsack (DP) — Pick best items

under limit

Only Required For Interviews

Advanced DSA

— KMP Algo — Fast string matching

— Z-Algorithm — Find patterns in strings

— Mo's Algorithm Speed up range queries

Segment Tree - Range query + update

— Fenwick Tree (BIT) — Like segment tree,
but smaller

— Suffix Array — Sort substrings smartly

— Tarjan's Algo Find critical links in graph

— Bitmask DP — Use bits to track states

Matrix Exponentiation Fast Fibonacci

& recursions

Sieve + Mod Power — Prime tricks in
number theory

Additionally

Required For competitive Programming

Math for DSA (More Than
Enough)

Prime Numbers — Sieve, factorization

— GCD & LCM Euclid's algorithm

Modulo Math Use % smartly (for big
numbers)

— nCr & Combinations Count ways to
choose

— Bit Manipulation — use bits to solve
problems

Power & Exponentiation — Fast pow
(modular)

Log & Binary Math For complexity, bit
length

— Probability & Counting Basic
combinatorics

— Math Patterns — Series, squares, sums

— Divisors & Multiples — Common trick in
number problems

Maths For Both

COMPANY WISE —DSA PROBLEM LINK

<https://github.com/krishnadey30/LeetCode-Questions-CompanyWise>

[Common DSA Coding Patterns Asked in Interviews.pdf - Google Drive](#)



Time you can take:

Beginner (Little or No Prior Experience): Around 10-12 weeks

- Time per question: 1.5–2 hours
- Total time: ~150 hours
- Daily study: 2 hours per day

Intermediate (Some Familiarity with DSA): Around 6-8 weeks

- Time per question: 1–1.5 hours
- Total time: ~100 hours
- Daily study: 2 hours per day

Advanced (Good Knowledge of DSA): Around 4-6 weeks

- Time per question: 0.75–1 hour
- Total time: ~65 hours
- Daily study: 2 hours per day

1. Arrays

- Two Sum - LeetCode #1
- Best Time to Buy and Sell Stock - LeetCode #121
- Product of Array Except Self - LeetCode #238
- Subarray Sum Equals K - LeetCode #560
- Merge Intervals - LeetCode #56
- Find Minimum in Rotated Sorted Array - LeetCode #153
- Max Consecutive Ones - LeetCode #485
- Contains Duplicate - LeetCode #217
- Majority Element - LeetCode #169

2. Strings

- Longest Substring Without Repeating Characters - LeetCode #3
- Valid Anagram - LeetCode #242
- Longest Palindromic Substring - LeetCode #5
- Group Anagrams - LeetCode #49
- Valid Parentheses - LeetCode #20
- Implement strStr() - LeetCode #28
- Longest Common Prefix - LeetCode #14
- Count and Say - LeetCode #38

3. Linked List

- Reverse Linked List - LeetCode #206
- Merge Two Sorted Lists - LeetCode #21
- Linked List Cycle - LeetCode #141
- Remove Nth Node From End of List - LeetCode #19
- Reorder List - LeetCode #143
- Add Two Numbers - LeetCode #2
- Intersection of Two Linked Lists - LeetCode #160

4. Stack

- Valid Parentheses - LeetCode #20
- Min Stack - LeetCode #155
- Evaluate Reverse Polish Notation - LeetCode #150
- Largest Rectangle in Histogram - LeetCode #84
- Simplify Path - LeetCode #71
- Next Greater Element I - LeetCode #496

5. Queue

- Implement Queue using Stacks - LeetCode #232
- Sliding Window Maximum - LeetCode #239
- Design Circular Queue - LeetCode #622
- Moving Average from Data Stream - LeetCode #346
- Number of Recent Calls - LeetCode #933
- Rotten Oranges - LeetCode #994

6. Tree

- Invert Binary Tree - LeetCode #226
- Maximum Depth of Binary Tree - LeetCode #104
- Binary Tree Level Order Traversal - LeetCode #102
- Symmetric Tree - LeetCode #101
- Lowest Common Ancestor of a Binary Search Tree - LeetCode #235
- Construct Binary Tree from Preorder and Inorder Traversal - LeetCode #105
- Serialize and Deserialize Binary Tree - LeetCode #297

7. Graph

- Clone Graph - LeetCode #133
- Number of Connected Components in an Undirected

S.No	Topic	Problem Name	Difficulty Rating
1	Arrays	Contains Duplicate	5
2	Arrays	Valid Anagram	5
3	Two Pointers	Valid Palindrome	5
4	Sliding Window	Best Time to Buy And Sell Stock	5
5	Stack	Valid Parentheses	5
6	Binary Search	Binary Search	5
7	Linked List	Reverse Linked List	5
8	Trees	Invert Binary Tree	5
9	Trees	Same Tree	5
10	1-D DP	Climbing Stairs	5
11	Math	Plus One	5
12	Arrays	Two Sum	10
13	Two Pointers	Two Sum II Input Array Is Sorted	10
14	Stack	Min Stack	10
15	Linked List	Merge Two Sorted Lists	10
16	Trees	Maximum Depth of Binary Tree	10
17	Priority Queue	Kth Largest Element In a Stream	10
18	Backtracking	Subsets	10
19	1-D DP	Min Cost Climbing Stairs	10
20	Math	Happy Number	10
21	Bit Manipulation	Number of 1 Bits	10
22	Binary Search	Search a 2D Matrix	20
23	Linked List	Linked List Cycle	20
24	Trees	Diameter of Binary Tree	20
25	Priority Queue	Last Stone Weight	20
26	1-D DP	House Robber	20
27	2-D DP	Unique Paths	20
28	Greedy	Maximum Subarray	20
29	Intervals	Meeting Rooms	20
30	Math	Set Matrix Zeros	20
31	Bit Manipulation	Single Number	20
32	Trees	Balanced Binary Tree	25
33	Graphs	Max Area of Island	25
34	1-D DP	House Robber II	25
35	Greedy	Jump Game	25
36	Math	Rotate Image	25
37	Bit Manipulation	Counting Bits	25
38	Bit Manipulation	Missing Number	25
39	Arrays	Group Anagrams	35
40	Sliding Window	Longest Substring Without Repeating Characters	35
41	Stack	Evaluate Reverse Polish Notation	35
42	Binary Search	Koko Eating Bananas	35
43	Linked List	Reorder List	35
44	Trees	Subtree of Another Tree	35
45	Priority Queue	K Closest Points to Origin	35
46	Backtracking	Combination Sum	35
47	Backtracking	Permutations	35
48	Backtracking	Letter Combinations of a Phone Number	35
49	Graphs	Number of Islands	35
50	Graphs	Walls And Gates	35
51	Graphs	Rotting Oranges	35

Graph - LeetCode #323

- Course Schedule - LeetCode #207
- Word Ladder - LeetCode #127
- Pacific Atlantic Water Flow - LeetCode #417
- Minimum Number of Vertices to Reach All Nodes - LeetCode #1557
- Shortest Path in Binary Matrix - LeetCode #1091

8. Hashing

- Two Sum - LeetCode #1
- Subarray Sum Equals K - LeetCode #560
- Longest Substring Without Repeating Characters - LeetCode #3
- Group Anagrams - LeetCode #49
- Top K Frequent Elements - LeetCode #347
- Contains Duplicate - LeetCode #217
- Minimum Window Substring - LeetCode #76

9. Heap

- Kth Largest Element in an Array - LeetCode #215
- Merge k Sorted Lists - LeetCode #23
- Top K Frequent Elements - LeetCode #347
- Find Median from Data Stream - LeetCode #295
- Sliding Window Maximum - LeetCode #239
- Minimize Deviation in Array - LeetCode #1675

10. Recursion and Backtracking

- Subsets - LeetCode #78
- Combination Sum - LeetCode #39
- Permutations - LeetCode #46
- N-Queens - LeetCode #51
- Word Search - LeetCode #79
- Generate Parentheses - LeetCode #22
- Sudoku Solver - LeetCode #37

11. Dynamic Programming (DP)

- Climbing Stairs - LeetCode #70
- House Robber - LeetCode #198
- Coin Change - LeetCode #322
- Longest Increasing Subsequence - LeetCode #300
- Longest Common Subsequence - LeetCode #1143
- Edit Distance - LeetCode #72
- Unique Paths - LeetCode #62
- Partition Equal Subset Sum - LeetCode #416

• Practice Link

- <https://www.codingninjas.com/codestudio/guided-paths/data-structures-algorithms>
- Assumptions:

1. You know the basics of a coding language.
2. You have 45 days of summer break.

• Arrays & Strings

- Basic Array And Strings Questions
- Kadane's Algorithm
- Dutch National Flag Algorithm
- Sliding Window
- Two pointers

• Multidimensional arrays

- Traversal Based Problems
- Rotation Based Problems

• Recursion And Backtracking

- Basic Recursion Questions
- Divide And Conquer

• Sorting Algorithms

- Insertion Sort
- Selection Sort

• Binary Search Applications

S.No	Topic	Problem Name	Difficulty Rating
52	1-D DP	Longest Palindromic Substring	35
53	2-D DP	Longest Common Subsequence	35
54	Greedy	Gas Station	35
55	Intervals	Merge Intervals	35
56	Math	Spiral Matrix	35
57	Bit Manipulation	Reverse Bits	35
58	Bit Manipulation	Reverse Integer	35
59	Arrays	Top K Frequent Elements	45
60	Stack	Generate Parentheses	45
61	Binary Search	Find Minimum In Rotated Sorted Array	45
62	Linked List	Remove Nth Node From End of List	45
63	Trees	Lowest Common Ancestor of a Binary Search Tree	45
64	Priority Queue	Kth Largest Element In An Array	45
65	Backtracking	Combination Sum II	45
66	Backtracking	Subsets II	45
67	Graphs	Surrounded Regions	45
68	1-D DP	Palindromic Substrings	45
69	2-D DP	Best Time to Buy And Sell Stock With Cooldown	45
70	Greedy	Jump Game II	45
71	Intervals	Insert Interval	45
72	Math	Pow(x,n)	45
73	Bit Manipulation	Sum of Two Integers	45
74	Arrays	Encode and Decode Strings	50
75	Two Pointers	3Sum	50
76	Sliding Window	Longest Repeating Character Replacement	50
77	Stack	Daily Temperatures	50
78	Binary Search	Search In Rotated Sorted Array	50
79	Linked List	Copy List With Random Pointer	50
80	Trees	Binary Tree Level Order Traversal	50
81	Priority Queue	Task Scheduler	50
82	Backtracking	Word Search	50
83	Tries	Implement Trie Prefix Tree	50
84	Graphs	Clone Graph	50
85	1-D DP	Decode Ways	50
86	2-D DP	Coin Change II	50
87	Greedy	Hand of Straights	50
88	Intervals	Non Overlapping Intervals	50
89	Intervals	Meeting Rooms II	50
90	Arrays	Product of Array Except Self	60
91	Two Pointers	Container With Most Water	60
92	Sliding Window	Permutation In String	60
93	Linked List	Add Two Numbers	60
94	Trees	Binary Tree Right Side View	60
95	Backtracking	Palindrome Partitioning	60

- Binary Search On Arrays
- Binary Search On Matrix
- Linked Lists
 - Reversal Problems
 - Sorting Problems
 - Slow And Fast Pointers
 - Modify In Linked list
- Stacks & Queues
 - Implementation Based Problems
 - Application Based Problems
- Binary Trees
 - Tree Traversals
 - Construction Of Trees
 - Tree Views
 - Standard Problems
- BST
 - Construction Of BST
 - Conversion Based Problems
 - Modification in BST
 - Standard Problems
- Priority Queues And Heaps
 - Implementation Based problems
 - Conversion based problems
 - K Based Problems
- Graphs
 - Graph Traversals - BFS And DFS
 - MST
 - Shortest Path Algorithms
 - Topological Sort
 - Graphs in Matrix
- Dynamic Programming
 - DP with Arrays
 - DP With Strings
 - DP With Maths
 - DP With Trees
 - Breaking And Partition Based Problems
 - Counting Based Problems
- Hard Recursion And Backtracking Questions
- Other Topics
 - Hashmaps
 - Tries
 - Bit Manipulation
 - Greedy
 - Circular Queues
 - Deques - Hot Topic
 - Doubly And Circular LL
 - String Algorithms like KMP and Z Algorithm

Company Name	DSA Problem Link	Tricks for Solving Patterns
Google	Google DSA Problems	Focus on backtracking and dynamic programming.

S.No	Topic	Problem Name	Difficulty Rating
96	Graphs	Pacific Atlantic Water Flow	60
97	1-D DP	Coin Change	60
98	2-D DP	Target Sum	60
99	Greedy	Merge Triplets to Form Target Triplet	60
100	Math	Multiply Strings	60
101	Arrays	Valid Sudoku	70
102	Sliding Window	Minimum Window Substring	70
103	Stack	Car Fleet	70
104	Binary Search	Time Based Key Value Store	70
105	Linked List	Find The Duplicate Number	70
106	Trees	Count Good Nodes In Binary Tree	70
107	Trees	Validate Binary Search Tree	70
108	Priority Queue	Design Twitter	70
109	Tries	Design Add And Search Words Data Structure	70
110	Graphs	Course Schedule	70
111	Graphs	Graph Valid Tree	70
112	1-D DP	Maximum Product Subarray	70
113	1-D DP	Word Break	70
114	2-D DP	Interleaving String	70
115	Greedy	Partition Labels	70
116	Greedy	Valid Parenthesis String	70
117	Intervals	Minimum Interval to Include Each Query	70
118	Math	Detect Squares	70
119	Sliding Window	Sliding Window Maximum	75
120	Linked List	LRU Cache	75
121	Trees	Kth Smallest Element In a BST	75
122	Graphs	Course Schedule II	75
123	Graphs	Number of Connected Components In An Undirected Graph	75
124	Graphs	Redundant Connection	75
125	Advanced Graphs	Network Delay Time	75
126	1-D DP	Longest Increasing Subsequence	75
127	2-D DP	Longest Increasing Path In a Matrix	75
128	Arrays	Longest Consecutive Sequence	85
129	Two Pointers	Trapping Rain Water	85
130	Stack	Largest Rectangle In Histogram	85
131	Binary Search	Median of Two Sorted Arrays	85
132	Linked List	Merge K Sorted Lists	85
133	Trees	Construct Binary Tree From Preorder And Inorder Traversal	85
134	Priority Queue	Find Median From Data Stream	85
135	Backtracking	N Queens	85
136	Tries	Word Search II	85

Microsoft	Microsoft DSA Problems	Practice binary search, greedy algorithms.
Amazon	Amazon DSA Problems	Work on sliding window, heap, and DFS/BFS.
Facebook	Facebook DSA Problems	Master recursion, graphs, and bit manipulation.
Apple	Apple DSA Problems	Prioritize dynamic programming and divide and conquer.
Adobe	Adobe DSA Problems	Practice sorting and searching techniques.
Goldman Sachs	Goldman Sachs DSA Problems	Focus on arrays, hashing, and string manipulation.
Uber	Uber DSA Problems	Work on graphs, BFS/DFS, and Dijkstra's algorithm.
LinkedIn	LinkedIn DSA Problems	Focus on two-pointer and sliding window techniques.
Netflix	Netflix DSA Problems	Prioritize dynamic programming and DP on trees.
Twitter	Twitter DSA Problems	Work on greedy algorithms and interval problems.
Dropbox	Dropbox DSA Problems	Master recursion and binary search on sorted arrays.
Airbnb	Airbnb DSA Problems	Practice graph traversal and backtracking.
Salesforce	Salesforce DSA Problems	Work on greedy algorithms and dynamic programming.
Oracle	Oracle DSA Problems	Focus on matrix traversal and dynamic programming.
PayPal	PayPal DSA Problems	Prioritize binary search and sorting problems.
Walmart	Walmart DSA Problems	Focus on hashing, prefix sums, and arrays.
Expedia	Expedia DSA Problems	Practice stack-based problems and recursion.
Snap	Snap DSA Problems	Focus on graphs and dynamic programming.
Yahoo	Yahoo DSA Problems	Work on linked lists and recursion.
DoorDash	DoorDash DSA Problems	Master binary trees and backtracking.
Stripe	Stripe DSA Problems	Practice greedy algorithms and string manipulation.
Lyft	Lyft DSA Problems	Focus on two-pointer and sliding window techniques.
Intuit	Intuit DSA Problems	Work on backtracking and dynamic programming.
IBM	IBM DSA Problems	Master dynamic programming and recursion problems.
Atlassian	Atlassian DSA Problems	Focus on graph traversal and dynamic programming.
Reddit	Reddit DSA Problems	Work on hashing and bit manipulation.
Pinterest	Pinterest DSA Problems	Master recursion and divide and conquer techniques.
Spotify	Spotify DSA Problems	Focus on sorting, searching, and heaps.
Bloomberg	Bloomberg DSA Problems	Work on arrays, dynamic programming, and graphs.
Cisco	Cisco DSA Problems	Focus on linked lists and dynamic programming.
ByteDance	ByteDance DSA Problems	Master sorting algorithms and binary search.
Tesla	Tesla DSA Problems	Work on graph traversal and dynamic programming.
TikTok	TikTok DSA Problems	Prioritize dynamic programming and recursion.
Nvidia	Nvidia DSA Problems	Practice bit manipulation and backtracking.

S.No	Topic	Problem Name	Difficulty Rating
137	Graphs	Word Ladder	85
138	Advanced Graphs	Reconstruct Itinerary	85
139	1-D DP	Partition Equal Subset Sum	85
140	2-D DP	Distinct Subsequences	85
141	2-D DP	Edit Distance	85
142	Linked List	Reverse Nodes In K Group	95
143	Trees	Binary Tree Maximum Path Sum	95
144	Advanced Graphs	Min Cost to Connect All Points	95
145	Advanced Graphs	Cheapest Flights Within K Stops	95
146	2-D DP	Burst Balloons	95
147	Trees	Serialize And Deserialize Binary Tree	100
148	Advanced Graphs	Swim In Rising Water	100
149	Advanced Graphs	Alien Dictionary	100
150	2-D DP	Regular Expression Matching	100

- 1 Question (Problem Statement)
- 2 Constraints & Observations
- 3 Approach & Thought Process
- 4 Different Approaches (Brute Force → Optimized)
- 5 Example & Dry Run
- 6 Code (Clean + Commented)
- 7 TC & SC Analysis (Explained in Detail)
- 8 Code Walkthrough (Step-by-Step Execution)
- 9 Edge Cases
- 10 Mistakes to Avoid
- 1 1 Related Problems / Concepts / Patterns
- 1 2 Real-World Applications (if applicable)

solve and note and track this way ---you will improve for sure

👉 CONTESTS

Contest - LeetCode

Enhance your coding abilities and get valuable real-world feedback by participating in contests on LeetCode. You
<https://leetcode.com/contest/>



Coding Ninjas Studio

<https://www.codingninjas.com/studio/contests/weekly-contest-115>

Competitive Programming and Coding Challenges - CodeChef

Practice in the CodeChef monthly coding contests, and master competitive programming. Get all the information about the rated coding challenges hosted every week.
<https://www.codechef.com/contests>



Codeforces

Codeforces. Programming competitions and contests, programming community
<https://codeforces.com/>





PROBLEM OF THE DAY POTD—LEETCODE

Power of Three - LeetCode

Can you solve this real interview question? Power of Three - Given an integer n , return true if it is a power of three. Otherwise, return false. An integer n is a power of three, if there exists an integer x such that $n == 3^x$. Example 1: Input: $n = 27$ Output: true Explanation: $27 = 3^3$ Example 2: Input: $n = 0$ Output: false Explanation: There is no x where

<https://leetcode.com/problems/power-of-three/?envType=daily-question&envId=2025-08-13>



Coding Ninjas Studio

<https://www.codingninjas.com/studio/problem-of-the-day>

Problem of the Day | GeeksforGeeks | A computer science portal for geeks

Solve one problem based on Data Structures and Algorithms every day and win exciting prizes.

<https://www.geeksforgeeks.org/problem-of-the-day>



goutham sankeerth

👉 17 3-Month DSA Study Plan-just do!

A **Three Months** Data Structure and Algorithms Roadmap for all Levels of preparations.

Preparation Level

📅 Everyday for **3 Months**

	Daily Min Hours	Daily Min Score	Ready
Basic	1 Hr	200	70%
Serious	3 Hr	400	95%
Insane	5 Hr	600	99%

Pre-requisites:

- Working knowledge of Any Object-oriented Programming Language. Preferably C++, Java or Python.

▼ Month 1: Foundation

▼ Pre-requisites

- Have a basic understanding of Arrays, Linked Lists, Stack and Queues

▼ Problems

Rating	Topic	Easy (100)	Medium (200)	Hard (400)	Patterns
5 ⭐	Arrays	10	6	3	1. Two Pointers Algorithm 2. Sorting Based Problems 3. Kadane's Algorithm 4. Array Rotations 5. Subarrays Based Problems
4 ⭐	Linked List	8	4	2	1. Nodes Manipulation 2. Reversing, Reordering List 3. Manipulating Two Lists
4 ⭐	Sorting	3	3	1	1. Merge Sort 2. Quick Sort 3. Inversion count
3 ⭐	Stacks	8	3	2	1. Implementation using Stack 2. Infix, Prefix, Postfix 3. Previous smaller element 4. Window-based problems
3 ⭐	Queues	6	3	1	1. Queue using Stack and vice-versa 2. Sliding window-based problems
3 ⭐	Backtracking	3	4	2	1. Knights' Tour 2. Rat in a Maze 3. N-Queen Problem
2 ⭐	Recursion	6	3	2	1. nCr, Josephus 2. Generating Palindrome Strings, Subsets of String

▼ Additional

- Keep preparing Notes and breakdown of important Problems and concepts.
- Beginners should focus more on the High Rated Topics.

▼ Month 2: Important Concepts

▼ Pre-requisites

- Revise the Notes and important concepts from the previous month
- Have a basic understanding of Hashing, Binary Searching and Binary Trees

▼ Problems

Rating	Topic	Easy (100)	Medium (200)	Hard (400)	Patterns
5 ⭐	Hashing	10	6	2	1. Substring, subarray based Hashing 2. String and Matrix-based Hashing Problems
4 ⭐	Binary Searching	6	3	2	1. Sorted and rotated arrays 2. square root, median of array 3. Using a middle point to solve the array problems like the Tree Cutter Problem
4 ⭐	Binary Tree	8	4	2	1. DFS Traversal of Binary Tree 2. Problems using BFS Traversal of Binary Tree 3. Upward Traversal based Problems.
4 ⭐	Binary Search Trees	4	5	2	1. Implementation of BST 2. Sorted Array Problem in

Rating	Topic	Easy (100)	Medium (200)	Hard (400)	Patterns
					BST, like Two-sum or Minimum Absolute Difference
2★	Priority Queue	3	4	1	1. Heap Implementations 2. Merge k-sorted Lists 3. Median or running stream
2★	Bit Manipulation	6	2	1	1. Bit masking 2. Problems using XOR properties
2★	Mathematics	6	4	1	1. problems using Euclid GCD 2. Modulo Arithmetics, Catalan's Numbers, Fermat's Little Theorem, Pigeonhole principle

▼ Additional

- Keep preparing Notes and breakdown of important Problems and concepts.
- Solve more Medium level problems and don't spend too much time on one problem.
- Start participating in mock interviews with your friends, Do pair programming.

▼ Month 3: Advanced Topics and Mock Tests

▼ Pre-requisites

- Revise the Notes and important concepts from the previous months.
- Have a basic understanding of Dynamic Programming and Graphs

▼ Problems

Rating	Topic	Easy (100)	Medium (200)	Hard (400)	Patterns
5★	Mock Tests	2	4	2	1. Practice company wise Mock Tests 2. Participate in Coding contests.
4★	Greedy Programming	3	5	2	1. Candy Distribution based Problems 2. Activity Selection based Problems
4★	Dynamic Programming	10	8	4	1. Linear DP 2. LCS Based DP 3. LIS Based DP 4. 0/1 Knapsack based DP 5. Grid-based DP 6. Partitioning DP
3★	Graphs	4	6	2	1. Grid-based problems like Capture region on board, water flow Problems 2. BFS and DFS based problems
2★	Disjoint Sets	1	4	2	1. Implementation using Path compression and rank
2★	Trie	1	3	2	1. Implementation 2. Working with n Strings, Sorting or finding common prefix.
2★	Segment Trees	0	3	1	1. Basic Implementation 2. N-Ary Rooted Tree Implementation
1★	String Algorithms	0	1	1	1. KMP Algorithm 2. Rabin Karp Algorithm

▼ Additional

- Beginners should focus more on solving Medium Level Problems and High-Rated Topics in the previous two months.
- Participate in Online Coding Contests and build your Coding Speed.
- Go through Previous Interview Experience.
- Dedicated preparation for the Aptitude Test is necessary. 7 Tests for 7 days, 30 minutes each.
- Practice Practice Practice...



6-Month DSA Study Plan-just do!

This structured six-month plan gradually introduces core DSA concepts and patterns, with 8-10 carefully selected practice questions each week. By the end of this plan, you'll have covered foundational topics and advanced concepts needed for top-tier technical interviews.

Special Resources

▼ Month 1: Arrays and Strings

Week 1: Arrays Basics & Simple Patterns

- **Topics:** Two-pointer, Sliding Window, HashMap basics
- **Patterns:** Prefix Sum, Running Sum
- **Practice:**

- ☐ [Easy] [Two Sum](#)
- ☐ [Easy] [Contains Duplicate](#)
- ☐ [Easy] [Maximum Subarray \(Kadane's Algorithm\)](#)
- ☐ [Medium] [Best Time to Buy and Sell Stock](#)
- ☐ [Medium] [Product of Array Except Self](#)
- ☐ [Medium] [Subarray Sum Equals K](#)
- ☐ [Medium] [Longest Subarray with Sum K](#)
- ☐ [Medium] [Minimum Size Subarray Sum](#)

Week 2: Advanced Array Manipulations

- **Topics:** Sorting, Interval Overlap, Cyclic Sort
- **Patterns:** Sorting + Two-pointer, Merge Intervals
- **Practice:**

- ☐ [Easy] [Sort Colors \(Dutch National Flag Problem\)](#)
- ☐ [Medium] [Merge Intervals](#)
- ☐ [Medium] [Set Matrix Zeroes](#)
- ☐ [Medium] [Next Permutation](#)
- ☐ [Medium] [Rotate Image](#)
- ☐ [Medium] [Find the Duplicate Number](#)
- ☐ [Medium] [Kth Largest Element in an Array](#)
- ☐ [Medium] [First Missing Positive](#)

Week 3: Strings - Basics and Manipulations

- **Topics:** HashMaps for Strings, Substrings
 - **Patterns:** Sliding Window, Hashing
 - **Practice:**
- ☐ [Easy] [Longest Common Prefix](#)
 - ☒ [Easy] [Valid Anagram](#)
 - ☐ [Medium] [Longest Substring Without Repeating Characters](#)
 - ☐ [Medium] [Minimum Window Substring](#)
 - ☐ [Medium] [Find All Anagrams in a String](#)
 - ☐ [Medium] [Group Anagrams](#)
 - ☐ [Medium] [Longest Palindromic Substring](#)
 - ☐ [Medium] [Decode Ways](#)

Week 4: Advanced String Manipulations

- **Topics:** Substrings and Subsequence
 - **Patterns:** Two-pointer, Dynamic Programming (DP) for Strings
 - **Practice:**
- ☐ [Easy] [Implement strStr\(\)](#)
 - ☐ [Medium] [Longest Common Subsequence](#)
 - ☐ [Medium] [Edit Distance](#)
 - ☐ [Medium] [Palindromic Substrings](#)
 - ☐ [Medium] [Partition Labels](#)
 - ☐ [Hard] [Minimum Window Substring](#)

☐ [Hard] [Substring with Concatenation of All Words](#)

☐ [Hard] [Wildcard Matching](#)

▼ Month 2: Linked Lists and Binary Trees

Week 1: Linked Lists Basics

- **Topics:** Singly and Doubly Linked Lists, Basic Operations
- **Patterns:** Two-pointer, Fast & Slow Pointers
- **Practice:**
 1. [Easy] [Reverse Linked List](#)
 2. [Easy] [Middle of the Linked List](#)
 3. [Medium] [Linked List Cycle](#)
 4. [Medium] [Intersection of Two Linked Lists](#)
 5. [Medium] [Add Two Numbers](#)
 6. [Medium] [Remove N-th Node from End](#)
 7. [Medium] [Flatten a Multilevel Doubly Linked List](#)
 8. [Hard] [Merge K Sorted Lists](#)

Week 2: Advanced Linked Lists

- **Topics:** Complex List Manipulations, Reversal in Groups
- **Patterns:** Fast & Slow Pointers, Recursive Linked List
- **Practice:**
 1. [Medium] [Reverse Nodes in k-Group](#)
 2. [Medium] [Rotate List](#)
 3. [Medium] [Copy List with Random Pointer](#)
 4. [Medium] [Partition List](#)
 5. [Medium] [LRU Cache](#)
 6. [Hard] [LRU Cache](#)
 7. [Hard] [Reverse Nodes in k-Group](#)
 8. [Hard] [Insert into a Cyclic Sorted List](#)

Week 3: Binary Trees Basics

- **Topics:** Binary Tree Traversals (Inorder, Preorder, Postorder)
- **Patterns:** DFS and BFS Traversals
- **Practice:**
 1. [Easy] [Binary Tree Inorder Traversal](#)
 2. [Easy] [Binary Tree Preorder Traversal](#)
 3. [Easy] [Binary Tree Postorder Traversal](#)
 4. [Medium] [Binary Tree Level Order Traversal](#)
 5. [Medium] [Binary Tree Zigzag Level Order Traversal](#)
 6. [Medium] [Binary Tree Right Side View](#)
 7. [Medium] [Binary Tree Left Side View](#)
 8. [Medium] [Binary Tree Maximum Path Sum](#)

Week 4: Advanced Trees

- **Topics:** Tree Properties, Recursive & Iterative Solutions
- **Patterns:** Depth-First Search, Breadth-First Search
- **Practice:**
 1. [Easy] [Symmetric Tree](#)
 2. [Medium] [Lowest Common Ancestor of a Binary Tree](#)
 3. [Medium] [Binary Tree Level Order Traversal II](#)
 4. [Medium] [Serialize and Deserialize Binary Tree](#)
 5. [Medium] [Populating Next Right Pointers in Each Node](#)
 6. [Hard] [Recover Binary Search Tree](#)
 7. [Hard] [Flatten Binary Tree to Linked List](#)
 8. [Hard] [Binary Tree Cameras](#)

▼ Month 3: Graphs and Recursion

Week 1: Graphs Basics

- **Topics:** Graph Representations, BFS, DFS
- **Patterns:** Breadth-First Search, Depth-First Search
- **Practice:**
 1. [Easy] [Flood Fill](#)
 2. [Medium] [Number of Islands](#)
 3. [Medium] [Surrounded Regions](#)
 4. [Medium] [Rotting Oranges](#)
 5. [Medium] [Word Ladder](#)
 6. [Medium] [Clone Graph](#)
 7. [Medium] [Course Schedule](#)
 8. [Medium] [Detect Cycle in Undirected Graph](#)
 9. [Hard] [Minimum Number of Swaps for Sorting](#)

Week 2: Advanced Graph Algorithms

- **Topics:** Topological Sort, Connected Components
- **Patterns:** Topological Sort, Union-Find
- **Practice:**
 1. [Medium] [Alien Dictionary](#)
 2. [Medium] [Course Schedule II](#)
 3. [Medium] [Reconstruct Itinerary](#)
 4. [Medium] [Connected Components in Graph](#)
 5. [Medium] [Accounts Merge](#)
 6. [Hard] [Find Critical and Pseudo-Critical Edges](#)
 7. [Hard] [Redundant Connection](#)
 8. [Hard] [Graph Valid Tree](#)

Week 3: Recursion and Backtracking

- **Topics:** Basics of Recursion, Permutations, Combinations
- **Patterns:** Backtracking, DFS
- **Practice:**
 1. [Easy] [Subsets](#)
 2. [Medium] [Combination Sum](#)
 3. [Medium] [Permutations](#)
 4. [Medium] [Palindrome Partitioning](#)
 5. [Hard] [N-Queens](#)
 6. [Hard] [Sudoku Solver](#)
 7. [Hard] [Word Search II](#)
 8. [Hard] [K-th Permutation Sequence](#)

Week 4: Advanced Backtracking Problems

- **Topics:** Constraint Satisfaction, Optimizations
- **Patterns:** Recursion + Memoization
- **Practice:**
 1. [Medium] [Generate Parentheses](#)
 2. [Medium] [Letter Combinations of a Phone Number](#)
 3. [Medium] [Partition to K Equal Sum Subsets](#)
 4. [Hard] [Restore IP Addresses](#)
 5. [Hard] [Expression Add Operators](#)
 6. [Hard] [Remove Invalid Parentheses](#)
 7. [Hard] [Count of Unique BSTs](#)
 8. [Hard] [Word Search](#)

▼ Month 4: Dynamic Programming (DP) - Part 1

Week 1: Introduction to Dynamic Programming

- **Topics:** Fundamentals of DP, 1D DP

- **Patterns:** Memoization, Tabulation
- **Practice:**
 1. [Easy] [Climbing Stairs](#)
 2. [Medium] [House Robber](#)
 3. [Medium] [House Robber II](#)
 4. [Medium] [Decode Ways](#)
 5. [Medium] [Maximum Subarray](#)
 6. [Medium] [Jump Game](#)
 7. [Medium] [Longest Increasing Subsequence](#)
 8. [Hard] [Longest Substring with K Distinct Characters](#)

Week 2: Intermediate DP

- **Topics:** 0/1 Knapsack and Variants
- **Patterns:** Knapsack, Subset Sum
- **Practice:**
 1. [Medium] [Partition Equal Subset Sum](#)
 2. [Medium] [Combination Sum IV](#)
 3. [Medium] [Coin Change](#)
 4. [Medium] [Target Sum](#)
 5. [Medium] [Ones and Zeroes](#)
 6. [Medium] [Minimum Subset Sum Difference](#)
 7. [Hard] [Word Break](#)

Week 3: Advanced DP - Sequence Alignment

- **Topics:** Longest Common Subsequence and Variants
- **Patterns:** Substring and Subsequence Problems
- **Practice:**
 1. [Medium] [Longest Common Subsequence](#)
 2. [Medium] [Longest Common Substring](#)
 3. [Medium] [Minimum Insertions/Deletions](#)
 4. [Medium] [Edit Distance](#)
 5. [Medium] [Longest Palindromic Subsequence](#)
 6. [Medium] [Sequence Pattern Matching](#)
 7. [Hard] [Palindrome Partitioning II](#)
 8. [Hard] [Distinct Subsequences](#)

Week 4: Advanced DP - Matrix and Grid Problems

- **Topics:** 2D DP, Grid Path Problems
- **Patterns:** Dynamic Programming on Grids
- **Practice:**
 1. [Easy] [Unique Paths](#)
 2. [Medium] [Unique Paths II](#)
 3. [Medium] [Minimum Path Sum](#)
 4. [Medium] [Dungeon Game](#)
 5. [Medium] [Cherry Pickup](#)
 6. [Hard] [Interleaving String](#)
 7. [Hard] [Maximal Rectangle](#)
 8. [Hard] [Largest Rectangle in Histogram](#)

▼ Month 5: Greedy Algorithms, Advanced Dynamic Programming, and Heaps

Week 1: Greedy Algorithms - Intermediate Problems

- **Topics:** Interval Scheduling, Partitioning, Activity Selection
- **Patterns:** Interval Problems, Greedy Choice Property
- **Practice:**
 1. [Easy] [Best Time to Buy and Sell Stock II](#)
 2. [Medium] [Jump Game](#)

3. [Medium] [Jump Game II](#)
4. [Medium] [Gas Station](#)
5. [Medium] [N Meetings in One Room](#)
6. [Medium] [Partition Labels](#)
7. [Medium] [Task Scheduler](#)
8. [Hard] [Minimum Number of Refueling Stops](#)

Week 2: Heaps - Basics and Applications

- **Topics:** Heap Basics, Priority Queue, Top-K Problems
- **Patterns:** Min-Heap, Max-Heap
- **Practice:**
 1. [Easy] [Kth Largest Element in a Stream](#)
 2. [Medium] [Kth Largest Element in an Array](#)
 3. [Medium] [Top K Frequent Elements](#)
 4. [Medium] [K Closest Points to Origin](#)
 5. [Medium] [Sort Characters By Frequency](#)
 6. [Hard] [Find Median from Data Stream](#)
 7. [Hard] [Merge K Sorted Lists](#)
 8. [Hard] [Sliding Window Maximum](#)

Week 3: Advanced Dynamic Programming - Optimization Problems

- **Topics:** DP on Trees, Knapsack Variants
- **Patterns:** 0/1 Knapsack, DP with Trees
- **Practice:**
 1. [Medium] [Partition Equal Subset Sum](#)
 2. [Medium] [Ones and Zeroes](#)
 3. [Medium] [Coin Change](#)
 4. [Medium] [Target Sum](#)
 5. [Medium] [Tree Diameter](#)
 6. [Hard] [Longest Increasing Path in a Matrix](#)
 7. [Hard] [Maximal Square](#)
 8. [Hard] [Burst Balloons](#)

Week 4: Advanced Greedy Algorithms & Special Topics

- **Topics:** Advanced Greedy Techniques, Miscellaneous
- **Patterns:** Greedy + Dynamic Programming, Hybrid Patterns
- **Practice:**
 1. [Medium] [Minimum Number of Arrows to Burst Balloons](#)
 2. [Medium] [Reduce Array Size to The Half](#)
 3. [Medium] [Divide Array in Sets of K Consecutive Numbers](#)
 4. [Hard] [Minimum Cost to Hire K Workers](#)
 5. [Hard] [IPO](#)
 6. [Hard] [Maximum Profit in Job Scheduling](#)
 7. [Hard] [Candy](#)

▼ Month 6: Tries, Advanced Data Structures, and Comprehensive Revision

Week 1: Tries and Prefix Trees

- **Topics:** Trie Basics, String Manipulations with Trie
- **Patterns:** Trie Structure, Prefix Matching
- **Practice:**
 1. [Easy] [Implement Trie \(Prefix Tree\)](#)
 2. [Medium] [Add and Search Word - Data Structure Design](#)
 3. [Medium] [Word Search II](#)
 4. [Medium] [Replace Words](#)
 5. [Medium] [Design Search Autocomplete System](#)
 6. [Hard] [Palindrome Pairs](#)

7. [Hard] [Concatenated Words](#)
8. [Hard] [Maximum XOR of Two Numbers in an Array](#)

Week 2: Segment Trees and Binary Indexed Trees

- **Topics:** Range Queries, Lazy Propagation
- **Patterns:** Segment Tree, Fenwick Tree
- **Practice:**
 1. [Medium] [Range Sum Query - Mutable](#)
 2. [Medium] [Range Sum Query 2D - Mutable](#)
 3. [Medium] [Count of Smaller Numbers After Self](#)
 4. [Medium] [Kth Largest Element in an Array](#)
 5. [Hard] [Reverse Pairs](#)
 6. [Hard] [The Skyline Problem](#)
 7. [Hard] [Maximum Sum of Rectangle No Larger Than K](#)

Week 3: Final Revision - Arrays, Strings, Linked Lists

- **Topics:** Comprehensive Review and Practice
- **Practice:**
 1. **Review** top 3-5 questions from Arrays and Strings sections
 2. **Review** top 3-5 questions from Linked Lists
 3. **Practice Mock Interviews** with a random mix of problems
 4. Focus on any weak areas or topics that need additional review

Week 4: Final Revision - Trees, Graphs, DP, and Advanced Topics

- **Topics:** Comprehensive Review and Practice
- **Practice:**
 1. **Review** top 3-5 questions from Trees and Graphs
 2. **Review** top 3-5 questions from Dynamic Programming
 3. **Practice Mock Interviews** with a random mix of problems
 4. Focus on polishing solutions for harder problems or any weak areas

🌟 Remember

"The best investment you can make is in yourself. Learning never goes out of style!"

Start today, stay consistent, and watch your career transform! 🚀

Happy Learning! Share this guide with friends who want to upgrade their skills!



Follow for More Resources & Guidance:

Goutham Sankeerth

- 📺 YouTube: <https://www.youtube.com/@LearnwithGoutham>
- 📷 Instagram: [@gouthamsankeerth](#)

All Resources & Links:

- [Goutham Sankeerth - Linktree](#)

Note: This document reflects personal experiences, insights from colleagues, friends, and various online resources (Google, YouTube, Reddit, etc.), formatted with AI assistance for better readability. Hope this helps you on your journey!

- Goutham Sankeerth